Centre for Mathematical Sciences, CVML, October 2026

Image Analysis, Assignment 4

1 Color correction of images

In this task you will investigate ways of changing and correcting colors in images.

1) In the file `task1_color.png` an image $\mathbf{g}$ is given. There is an undesirable color cast in the image. The correct colors are given in the input image $\mathbf{f}$ (saved in the file `task1_org.png`). However a number of pixels in this image are corrupted (black).

The color change between the two images is assumed to follow a linear transformation in RGB-space, i.e. for the non-corrupted pixels the change can be written

$$\mathbf{g}(i,j) = A\mathbf{f}(i,j), \tag{1}$$

where

$$\mathbf{g}(i,j) \in \mathbb{R}^3, \quad \mathbf{f}(i,j) \in \mathbb{R}^3, \tag{2}$$

and A is an unknown 3×3-matrix.

1) Write a robust estimation method based on RANSAC that finds the transformation matrix A. Transform the image $\mathbf{g}$ using this matrix to find an estimate $\hat{\mathbf{f}}$ of the original image.

2) Calculate the *PSNR*, *SSIM* metrics, and the perceptual ꟻLIP error between your estimate $\hat{\mathbf{f}}$ and the original $\mathbf{f}$. Also compute all error metrics between the original $\mathbf{f}$ and the corrupted $\mathbf{g}$. Report both the mean error on the whole image as well as on only the uncorrupted pixels (non-black, using masking in the output pixel-wise error maps that ꟻLIP and SSIM produce).

Python specific instructions

There is an outline of a script that solves the task given in `task1.py`.

For PSNR and SSIM metrics you may use Python implementations available from e.g. the module `scikit-image`, but make sure that you understand roughly how these metrics are calculated. You can install the ꟻLIP package using pip (see `https://github.com/NVlabs/flip` for details).

For the report: Describe your implemented methods and supply your code, the estimated A and the resulting output images. How many corresponding pixels do you need minimally in your RANSAC to estimate A? Also provide a table with all the computed error metrics.

2 Segmentation with Graph Cuts

You are given the task of segmenting out two heart chambers in an image by Graph-Cuts. In `heart_data.mat` you are given two vectors, with a number of intensities that have been observed inside the two chambers (`chamber_values`) and a number of intensities observed in the background (`background_values`). You are also given the image of Figure 1, denoted by f below.

We will in this task use an available implementation for solving the graph cut problem using max flow. Study the material in the lectures and also read the chapter on Graph Cut segmentation in Szelisky "Computer Vision: Algorithms and Applications", and modify the script according to the following tasks:

1) Assume that the pixel intensities in the two classes are generated by two different Gaussian distributions. In other words, for pixel i, the likelihoods of observing intensity f_i, $P(f_i \,|\, \text{chamber class})$ and $P(f_i \,|\, \text{background class})$, respectively, are Gaussians. Recall, a Gaussian is given by $P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$. Estimate the means and the standard deviations (μ and σ) for the chamber class and the background class, respectively, using the provided ground truth values in the data file.

2) Given the theoretical assumptions on the distributions (Gaussian), what is the negative log likelihoods for the two classes (it should be a simple expression, derived analytically, and you can ignore the term arising from the normalizing constant in the Gaussian)?. Use the statistical Mumford-Shah functional formulation, i.e. instead of using the formulation based on

$$E_0(\Gamma) = \int_{R_1} (a_1 - f)^2 dxdy + \int_{R_2} (a_2 - f)^2 dxdy + \nu|\Gamma|, \tag{3}$$

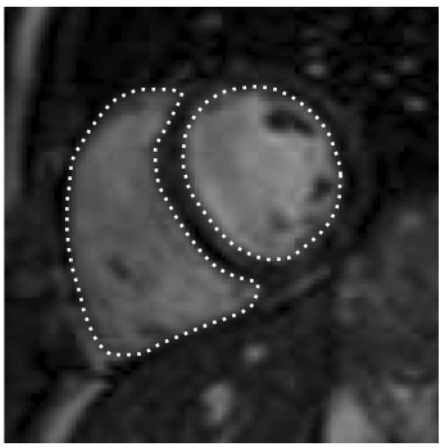


Figure 1: The data given in the task. A slice of a heart imaged by a MRI camera. This view is known as the "short-axis view" taken in a direction where the left and the right heart chambers are visible at the same time. The two chambers are shown inside the dotted lines, the left chamber is to the right in the image and vice versa.

use a data-term consisting of the negative log likelihoods for the two classes, in the sense of

$$E_0(\Gamma) = \int_{R_1} -\log P(f|\text{chamber class})dxdy + \int_{R_2} -\log P(f|\text{background class})dxdy + \nu|\Gamma| \tag{4}$$

Then construct a graph of the heart image with such data terms, and solve the corresponding discretized problem via max-flow/min-cut. Experiment with the prior/regularization weight ν, in order to obtain a reasonable segmentation. (Note: it is hard to get a perfect segmentation.)

3) Optional. Fine tune the segmentation by adding ground truth. This can be done by changing the data-term to ∞ (or a large number) for the pixels you know which region they belong to. This will force certain pixels to belong to a particular class. Such pixels can be obtained by manual inspection of the image.

Notes: Both 2) and 3) can be seen as adjustments a medical doctor could perform. In 2) a slider could be added to give interactive response to different weights. Then a pen tool can be used to introduce the ground truth of 3).

Python-specific instructions

In `task2.py` there is example code for setting up a graph and running the Scipy implementation of maximum flow. It uses a special sparse matrix format. There is also code for generating neighborhood indices for a given size of input image (`edges4/8connected`, don't change these functions!). Please modify the code (calculating the means and standard deviations of the given data, and setting up the weights for both the regularization term and the data term for graph cut segmentation, based on the statistical likelihood model).

For the report: supply your code, the estimated means and standard deviations, and a printout of the results of using your algorithm, i.e. supply input data (as an image) and result after applying your segmentation algorithm (also as an image).

3 Computer Vision

Assume that a camera matrix is given by

$$P_1 = \begin{pmatrix} 1 & 2 & 3 & 3 \\ 3 & 2 & 0 & -2 \\ 1 & 1 & 2 & 0 \end{pmatrix} .$$

Also assume that we have detected a point in the image at position $u = (6, -\frac{1}{2})$. Which of the following could be possible 3D points that project to the detected image point?

$$U_1 = (2, \frac{1}{2}, 1), \quad U_2 = (\frac{1}{3}, \frac{1}{3}, 0) \quad U_3 = (-12, 18, 1).$$

For the report: Provide your calculations, your answer and your motivation.

4 The Garment Classification system - part 4 - system construction and system testing

Here you will finish the work on your classification system. From the three previous assignments you have constructed

- `im2segment(Im)` - a segmentation algorithm that takes an image to a number of segments.
- `segment2features(S)` - an algorithm for calculating a feature vector x from a segment S.
- `features2class(x,classification_data)` - an algorithm for classifying a feature vector x as class y using machine learning.

The data files for this assignment contain both new test examples as well as a benchmark script. Run the benchmark script on all of the five datasets (short1, short2, home1, home2, home3). Write down the results before modifying your code. Then try improving your code to make it as good as possible.

- Test your system version 1 from Assignment 3 on the five datasets (short1, short2, home1, home2, home3).
- Try to change your system and produce a version 2 that works better on the five datasets.

Python-specific instructions

In `assignment4.py`, substitute the bad implementations of the three functions with your own. Then test your system on the five datasets, using e.g. the main script in `assignment4.py`.

Print the overall hitrate for version 1 on the five datasets in the report and describe with your own words the results in your report. If it went well, describe why. If it didn't go well describe why. Do the same for version 2 of your system. Describe what modifications you did in version 2 and why. You should aim at hitrates higher than 50% for all datasets. If you don't reach this level, describe what attempts you have made to increase the hitrate for version 2. Also include the Python code of your final system.